Data Structures and Algorithms — Lab 5 Report

Stack, Queue, Priority Queue, and Hash Table

Student: Cao Bao Khuong
Student ID: ITDSIU22176
Course: Data Structures and Algorithms
Lab: Lab 5 — Order-Based and Hash-Based Data Structures

Part A: Guided Practice Questions

Task A1 — Identify the Best Data Structure

#	Problem	Best Data Structure	Justification
1	Check whether a string of parentheses is valid	**Stack**	We need to match the most-recent opening bracket with the current closing bracket. LIFO order is exactly what we need.
2	Simulate a line of customers waiting for service	**Queue**	Customers are served in the order they arrived. FIFO is the natural model.
3	Find the top 3 most frequent numbers in an array	**HashMap + Priority Queue**	HashMap counts frequencies; a min-heap of size 3 keeps track of the 3 highest.
4	Count how many times each word appears in a paragraph	**HashMap**	Each word is a key; the count is its value. O(1) lookup and update.
5	Store student IDs and check if an ID already exists	**HashSet**	We only need membership checking (exists or not), no associated value needed.
6	Process tasks where each task has a different priority	**Priority Queue**	The task with the highest priority should be processed first, regardless of arrival order.
7	Implement undo and redo operations in a text editor	**Two Stacks**	One stack holds past actions (undo), the other holds undone actions (redo).

#	Problem	Best Data Structure	Justification
8	Store key-value pairs using your own hash table	**Hash Table (manual)**	Direct key-to-index mapping with collision handling via separate chaining.

Task A2 — What Is Stored?

I chose problems **1, 3, 4, and 7** to explain in detail.

Problem 1 — Valid Parentheses (Stack)

Data structure	Stack
What is stored	Opening bracket characters: `(`, `[`, `{`
When inserted	Every time we encounter an opening bracket
When removed	Every time we encounter a closing bracket — we pop and check if it matches
Why suitable	The most recently opened bracket must be the first one closed. Stack's LIFO order enforces this nesting rule naturally.

Problem 3 — Top 3 Most Frequent Numbers (HashMap + Priority Queue)

Data structure	HashMap for frequency counting, min-heap (Priority Queue) to track top-3
What is stored	HashMap: `{number → frequency}`. PriorityQueue: the 3 numbers with highest frequency so far
When inserted	HashMap: after reading each number. PriorityQueue: after computing all frequencies
When removed	PriorityQueue: when size exceeds 3, we remove the number with the lowest frequency
Why suitable	HashMap gives O(1) frequency lookup. The min-heap keeps the heap size bounded at 3, so finding the top-3 never costs more than O(n log 3) total.

Problem 4 — Word Frequency Count (HashMap)

Data structure	HashMap
What is stored	`{word → count}` pairs
When inserted	The first time we encounter a word; otherwise the count is updated
When removed	Not removed during counting — we only read at the end
Why suitable	Words map naturally to integer counts. HashMap's hash function gives O(1) average lookup, far better than scanning an array of pairs (O(n)).

Problem 7 — Undo / Redo in Text Editor (Two Stacks)

Data structure	Two Stacks: `undoStack` and `redoStack`
What is stored	States of the document (or action descriptions)
When inserted	`undoStack`: after every edit action. `redoStack`: after every undo
When removed	`undoStack.pop()` on Undo; `redoStack.pop()` on Redo. `redoStack` is cleared whenever a new edit is made
Why suitable	Each undo/redo needs the most recent action first — LIFO. Two stacks let us go back and forward without searching.

Part B: Required Exercises

Exercise B1 — Valid Parentheses

Problem Restatement

Given a string containing only `(`, `)`, `[`, `]`, `{`, `}`, determine whether every opening bracket has a correctly matched and correctly ordered closing bracket.

Input / Output / Assumptions / Edge Cases

- **Input:** A string of bracket characters
- **Output:** `true` if valid, `false` otherwise
- **Assumptions:** String contains only the 6 bracket characters

- **Edge cases:** empty string (valid), string with only closing brackets (invalid), unmatched opening brackets left at end (invalid)

Brute-Force Idea

Repeatedly scan the string and remove matched adjacent pairs like `()`, `[]`, `{}` until no more can be removed. If the string becomes empty it is valid. This is $O(n^2)$ because each pass is O(n) and there can be O(n/2) passes.

Optimized Idea (Stack)

Scan once left to right:

- If the character is an opening bracket → **push** it onto the stack
- If it is a closing bracket → **pop** the top of the stack and check if they match
- If the stack is empty when we try to pop → **invalid**
- After the full scan, if the stack is still non-empty → **invalid**

Selected Data Structure

Stack — because bracket matching is inherently LIFO: the most recently opened bracket must be the first one closed.

What Is Stored

The stack stores **opening bracket characters** that have been seen but not yet matched.

Why the Data Structure Is Suitable

A stack directly enforces the nesting rule. When we encounter `)` we need to check the *most recent* unmatched `(`, which is exactly what `stack.peek()` gives us in O(1).

Complexity

Time	O(n) — one pass through the string
Space	O(n) — worst case the stack holds every character (e.g., all opening brackets)

Test Cases

Input	**Expected**	**Actual**	**Notes**
`{[()]}`	`true`	`true`	Normal nested valid case
`{[(])}`	`false`	`false`	Wrong closing order
`""`	`true`	`true`	Edge: empty string
`]`	`false`	`false`	Edge: starts with closing bracket

Input	Expected	Actual	Notes
`((`	`false`	`false`	Edge: unclosed brackets
`()[]{}`	`true`	`true`	Multiple pairs, all valid

Exercise B2 — Implement a Circular Queue

Problem Restatement

Implement a queue using a fixed-size array where the front and rear pointers wrap around using modulo arithmetic, so that dequeued space can be reused. Must support: `enqueue`, `dequeue`, `front`, `isEmpty`, `isFull`.

Input / Output / Assumptions / Edge Cases

- **Input:** Integer values to enqueue; method calls
- **Output:** Return values of `dequeue()` and `front()`; boolean returns of `isEmpty()` / `isFull()`
- **Assumptions:** Fixed capacity set at construction
- **Edge cases:** Enqueue when full (overflow), dequeue when empty (underflow), wrap-around after rear reaches end of array

Optimized Idea (Circular Array)

Use four variables: `arr[]`, `front`, `rear`, `size`. When enqueuing:

```
arr[rear] = value
rear = (rear + 1) % capacity
size++
```

When dequeuing:

```
value = arr[front]
front = (front + 1) % capacity
size--
```

The modulo operation makes rear and front wrap back to index 0 when they hit the end, reusing freed slots.

Selected Data Structure

Array with circular indexing — manual implementation as required.

What Is Stored

The array stores the actual integer elements. `front` and `rear` are indices pointing to the oldest and next-empty slots respectively. `size` tracks how many elements are currently in the queue.

Why the Data Structure Is Suitable

A linear array would waste space (front moves right but old slots are never reused). Circular indexing reuses those slots by wrapping. This gives O(1) enqueue and dequeue without any shifting.

Why Circular Queue Is Better Than Simple Linear Queue

In a simple linear queue, after several enqueue/dequeue pairs, `front` advances right and space behind it is wasted. The queue "runs off the end" of the array even when there is capacity available. The circular queue solves this by treating the array as a ring.

Complexity

Time	O(1) for all operations
Space	O(capacity)

Test Cases

Operations	**Expected**
`enqueue(10), enqueue(20), enqueue(30)` then `isFull()`	`true`
`dequeue()` after above	`10`
`enqueue(40)` then `front()`	`20`
`dequeue()` on empty queue	underflow error, returns -1
`enqueue` on full queue	overflow error, returns false
wrap-around: fill 3-slot queue, dequeue once, enqueue 1 new, check order	dequeue order: 2 → 3 → 4

Exercise B3 — Top K Frequent Elements

Problem Restatement

Given an integer array `nums` and integer `k`, return the `k` most frequently occurring elements. Order of output does not matter.

Input / Output / Assumptions / Edge Cases

- **Input:** `int[] nums`, `int k` where `1 ≤ k ≤ number of distinct elements`
- **Output:** Array of `k` integers
- **Assumptions:** Answer is guaranteed to be unique (no tie-breaking needed)
- **Edge cases:** `k = 1`, all elements same frequency, single element

Brute-Force Idea

Count frequencies with a HashMap, convert to a list, sort by frequency descending, take the first `k`. Time: O(n log n) for sorting.

Optimized Idea (HashMap + Min-Heap)

1. Build `HashMap<Integer, Integer> freq` — map each number to its count
2. Iterate over distinct numbers, add each to a `PriorityQueue` ordered by frequency (smallest at top)
3. If the heap grows larger than `k`, remove the top (the least frequent)
4. After all numbers processed, the heap contains exactly the `k` most frequent elements

Selected Data Structure

HashMap for frequency counting + **PriorityQueue** (min-heap of size k) for selecting top-k.

What Is Stored

- HashMap: `{element → frequency}` — how many times each number appeared
- PriorityQueue: the `k` candidate elements ordered by their frequency

Why the Data Structure Is Suitable

HashMap gives O(1) insertion and lookup for frequency updates. The min-heap of size k means we never keep more than k elements in memory. When a new element is added, the element with the lowest frequency is immediately evicted. This avoids sorting the full frequency list.

What Determines Priority

The frequency (count) of each element in the HashMap. Lower frequency = higher priority for removal from the min-heap.

Why We Need Both Structures

HashMap alone can count frequencies but cannot efficiently find the top-k values. PriorityQueue alone cannot count frequencies. Together: HashMap computes what we need; PriorityQueue selects the best k from what HashMap found.

Complexity

Time	O(n log k) — n insertions, each heap operation costs O(log k)
Space	O(n) for HashMap + O(k) for heap

Test Cases

Input	k	Expected Output
`[1, 1, 1, 2, 2, 3]`	2	`[1, 2]`
`[1, 2, 2, 3, 3, 3]`	1	`[3]`
`[1, 2, 3]`	2	any 2 elements (all frequency = 1)
`[5]`	1	`[5]`

Exercise B4 — Implement a Simple Hash Table

Problem Restatement

Implement a hash table that maps `String` keys to `int` values using **separate chaining** for collision handling. Must support: `put`, `get`, `remove`, `containsKey`.

Input / Output / Assumptions / Edge Cases

- **Input:** String keys, integer values, method calls
- **Output:** `get` returns value or -1 if not found; `containsKey` returns boolean
- **Assumptions:** Table size is fixed (16 buckets)
- **Edge cases:** Collision (two keys hash to same index), updating an existing key, removing a key that does not exist, looking up a missing key

Optimized Idea

1. Compute the hash of the key using a polynomial rolling hash with multiplier 31
2. Take `Math.abs(hash) % TABLE_SIZE` to get the bucket index
3. Each bucket is a `LinkedList<Entry>` — this is the "separate chain"
4. For `put`: scan the chain for an existing entry with the same key (update it), or append a new entry
5. For `get` / `containsKey`: scan the chain for a matching key
6. For `remove`: remove the entry with matching key from the chain

Selected Data Structure

Array of LinkedLists — the array provides O(1) index access; the LinkedList at each index handles collisions.

What Is Stored

Each `Entry` holds a `String key` and `int value`. Each bucket (array slot) holds a `LinkedList<Entry>` — potentially empty, or containing one or more entries that happen to

hash to the same index.

How the Hash Value Is Computed

```
int h = 0;
for each character c in key:
    h = h * 31 + c
index = Math.abs(h) % TABLE_SIZE
```

Multiplying by 31 (a prime) at each step spreads different strings across the table well and avoids many collisions.

What Is a Collision

A collision occurs when two different keys produce the same table index. For example with `TABLE_SIZE = 10`: key `"ab"` and key `"ba"` might both hash to index 5.

How Separate Chaining Handles Collisions

Each bucket stores a linked list. When two keys collide, both are stored in the same bucket's list. Lookup scans the list comparing keys until a match is found.

Why Average Lookup Is Close to O(1)

If the hash function distributes keys evenly, each chain has on average `n / TABLE_SIZE` entries (the load factor). With a reasonable table size, this stays close to 1, making lookup effectively O(1) on average.

Complexity

Time (average)	O(1) for put, get, remove, containsKey
Time (worst case)	O(n) if all keys hash to the same bucket
Space	O(n) for n stored entries

Test Cases

Operations	**Expected**
`put("Alice", 90)`, `put("Alice", 95)`, `get("Alice")`	`95` (update works)
`put("Bob", 85)`, `containsKey("Bob")`	`true`
`remove("Bob")`, `containsKey("Bob")`	`false`
`get("Charlie")` (never inserted)	`-1`
Insert 20 keys into 16-slot table (forces collisions), then get all	All values correct

---

Part C: Further Practice Questions

I chose **C1** and **C3**.

Exercise C1 — Remove Adjacent Duplicates in a String

Problem Restatement

Given a string `s`, repeatedly remove pairs of adjacent identical characters until no adjacent pair exists. Return the final string.

Example: `"abbaca"` → remove `"bb"` → `"aaca"` → remove `"aa"` → `"ca"`

Input / Output / Assumptions / Edge Cases

- **Input:** String `s` of lowercase letters
- **Output:** Final string after all removals
- **Assumptions:** Characters are lowercase letters; result may be empty
- **Edge cases:** Single character, all same characters (even count → empty, odd count → one char), no adjacent duplicates at all

Brute-Force Idea

Scan the string repeatedly; each pass removes all adjacent pairs. Repeat until no more removals happen. This is O(n^2) in the worst case (e.g., `"aabb..."` has many passes).

Optimized Idea (Stack)

Scan each character once:

- If the stack is non-empty and `stack.peek() == currentChar` → **pop** (cancel the pair)
- Otherwise → **push** `currentChar`

After the scan, the stack contains only characters with no adjacent duplicates. Build the result string from the stack.

Selected Data Structure

Stack — because we always need to compare with the *most recently seen* character that hasn't been cancelled yet. Stack's LIFO property gives us this in O(1).

What Is Stored

Characters that have not yet been cancelled by a duplicate. The stack represents the "working result" — the string built so far with all duplicates removed up to the current position.

When We Push / When We Pop

- **Push:** when current character is different from the top (or stack is empty)

- **Pop:** when current character is the same as the top — they cancel each other

Why the Stack Avoids Repeated Scanning

Without a stack, we would need to re-scan the string after each removal because removing a pair can create a new adjacent pair. The stack naturally handles this: after a pop, the new top is the character just before the cancelled pair, so if the next character matches it, the pop happens immediately in the same pass.

Complexity

Time	O(n) — each character is pushed and popped at most once
Space	O(n) — worst case the stack holds every character

Test Cases

Input	**Expected**	**Notes**
`"abbaca"`	`"ca"`	Given example; two rounds of collapse
`"aabbcc"`	`""`	All pairs cancel out
`"abc"`	`"abc"`	No adjacent duplicates, nothing removed
`"a"`	`"a"`	Edge: single character
`"aaaa"`	`""`	Edge: all same, even count
`"abccba"`	`""`	Chain collapse: cc → bb → aa

Exercise C3 — Kth Largest Element in an Array

Problem Restatement

Given an integer array `nums` and integer `k`, return the kth largest element (not kth distinct). For example, in `[3, 2, 1, 5, 6, 4]` with `k=2`, the answer is `5`.

Input / Output / Assumptions / Edge Cases

- **Input:** `int[] nums`, `int k` where `1 ≤ k ≤ nums.length`
- **Output:** Single integer
- **Assumptions:** nums is non-empty, k is valid
- **Edge cases:** `k = 1` (maximum), `k = nums.length` (minimum), array with all duplicates

Brute-Force Idea

Sort the array in descending order and return `nums[k-1]`. Simple, but O(n log n).

Optimized Idea (Min-Heap of Size k)

Maintain a **min-heap** of at most `k` elements:

- Add each number to the heap
- If size exceeds `k`, remove the minimum (the smallest element in the heap)
- After all numbers are processed, the top of the heap is the kth largest

Selected Data Structure

PriorityQueue (min-heap) of size k.

Why a Min-Heap (Not Max-Heap)

We want to track the k largest elements seen so far. We use a min-heap so the **smallest among the top-k** is always at the top and can be cheaply removed when a larger element arrives. A max-heap would tell us the largest (rank 1), not the kth largest.

Why the Queue Size Is Limited to k

We only care about the top-k largest values. Keeping more than k elements wastes memory and slows down operations. By evicting the minimum whenever size > k, we always maintain exactly the k largest elements seen so far.

What the Top of the Heap Represents

The smallest element among the k largest elements seen so far. After processing all elements, this is exactly the kth largest in the full array.

Why This Is Better Than Sorting

Sorting: O(n log n) time, O(n) space. Min-heap approach: O(n log k) time, O(k) space.

When k << n (e.g., find the top 3 out of 1 million numbers), this is significantly faster.

Complexity

Time	O(n log k) — n insertions, each heap operation is O(log k)
Space	O(k) — heap never exceeds k elements

Test Cases

Input	**k**	**Expected**	**Notes**
`[3, 2, 1, 5, 6, 4]`	2	5	Given example

Input	k	Expected	Notes
`[3, 2, 3, 1, 2, 4, 5, 5, 6]`	1	`6`	k=1 is the maximum
`[1]`	1	`1`	Edge: single element
`[5, 3, 7, 1]`	4	`1`	k = length, result is minimum
`[2, 2, 2, 2]`	2	`2`	All duplicates

Part D: Reflection and Explanation

Task D1 — Concept Reflection

1. What is the difference between Stack and Queue?

Stack follows **LIFO** (Last-In, First-Out): the most recently added element is the first to be removed. Queue follows **FIFO** (First-In, First-Out): the oldest element is removed first. Stack is useful when the most recent state matters most (e.g., undo, bracket matching). Queue is useful when fairness/order of arrival matters (e.g., task scheduling, BFS).

2. What is the difference between Queue and Priority Queue?

A regular Queue removes elements strictly in the order they were inserted (FIFO). A Priority Queue removes elements based on a **priority value** — the element with the highest (or lowest) priority is removed first, regardless of insertion order. For example, a hospital emergency room uses priority (severity), not arrival order.

3. What is the difference between HashSet and HashMap?

HashSet stores only **keys** — it answers "does this item exist?" in O(1). HashMap stores **key-value pairs** — it answers "what value is associated with this key?" Both use hashing internally, but HashSet is for membership testing while HashMap is for mapping/counting/lookup.

4. What is a hash collision?

A collision happens when two different keys produce the same hash index. For example, with `TABLE_SIZE = 10`, keys `15` and `25` both map to index 5. Collisions are unavoidable when the number of possible keys exceeds the table size.

5. Why does a good hash function matter?

A poor hash function can map many keys to the same index, making all chains long and degrading lookup from O(1) to O(n). A good hash function distributes keys as evenly as possible across all buckets, keeping average chain length close to 1 and maintaining O(1) average performance.

6. Why is separate chaining useful?

Separate chaining handles collisions simply and robustly. Each bucket can hold any number of entries without needing to find another empty slot (as in open addressing). It never needs rehashing when the table fills, and deletion is straightforward (just remove from the linked list). The downside

is extra memory for the list pointers.

7. In Exercise B3, why do we need both HashMap and PriorityQueue?

HashMap alone can count frequencies but has no concept of ordering — we cannot efficiently get the "top k". PriorityQueue alone can order elements but cannot count frequencies. We need HashMap to compute what the frequency of each element is, then PriorityQueue to efficiently select the k elements with the highest frequencies.

8. Which required exercise was the hardest, and why?

Exercise B4 (implementing the hash table) was the hardest. Understanding how the hash function computes the index, then dealing with collisions via separate chaining, and making sure `put` correctly updates an existing key instead of adding a duplicate — all of these needed careful implementation. Getting the chain traversal right for all four operations (`put`, `get`, `remove`, `containsKey`) without mixing up references took several tries.

Task D2 — Explain Without Code (Exercise B3: Top K Frequent Elements)

Selected exercise: B3 — Top K Frequent Elements

Data structure used: HashMap + PriorityQueue (min-heap)

What information is stored:

The HashMap stores each distinct element paired with how many times it appeared in the array. The PriorityQueue stores at most `k` elements from the map, ordered so the element with the lowest frequency sits at the top.

When data is inserted or removed:

We insert into the HashMap as we read each number from the array — if the number is new, we add it with count 1; if it already exists, we increment its count. After the full array is processed, we insert each distinct number into the PriorityQueue. When the queue grows beyond size `k`, we immediately remove the element with the smallest frequency, since it cannot be in the top-k.

Why the structure is suitable:

This problem has two separate concerns: counting (HashMap is perfect — O(1) per update) and ranking (PriorityQueue is perfect — always gives access to the current minimum in O(log k)). Neither structure alone can do both; together they cover the problem efficiently.

Why the complexity is efficient:

Counting all frequencies takes O(n). Inserting each distinct element into the heap and maintaining size k takes O(m log k) where m is the number of distinct elements and m ≤ n. Overall O(n log k), which is much better than O(n log n) sorting — especially when k is small relative to n.

Maze Challenge — Algorithm Explanation

(Submitted separately as `StudentSolverImpl.java`)

Approach

The solver uses two phases:

SEARCH_RUN — Iterative DFS with backtracking

The mouse explores the maze by always moving to an **unvisited open neighbor**. When all neighbors of the current cell have been visited, it backtracks to the previous cell by popping from an explicit backtrack stack. At each cell, the open directions (and their neighbor positions) are recorded in a map so the learned maze structure persists between steps.

Since the simulator only accepts one action per call, turns and moves are **queued in advance**: the solver computes the sequence of `TURN_LEFT` / `TURN_RIGHT` / `MOVE_FORWARD` actions needed to reach the next cell and feeds them out one per call.

SPEED_RUN — BFS shortest path

After the SEARCH_RUN ends, the solver runs BFS on the map of open connections learned during exploration. BFS finds the shortest path (minimum number of cells) from the start to the goal. The mouse then follows this path directly.

Data Structures Used

Structure	Purpose
`Map<String, Map<Direction, int[]>>` openNeighbors	Stores open directions and neighbor positions for each visited cell
`Map<String, Boolean>` visited	Tracks which cells the DFS has explored
`Deque<int[]>` backStack	DFS backtrack stack — stores the physical path taken
`Deque<MouseAction>` actionQueue	Queued turn/move actions executed one per step
`Map<String, Direction>` deltaMap	Caches (Δrow, Δcol) → Direction for fast direction lookup
`List<int[]>` speedRunPath	BFS-computed shortest path for SPEED_RUN

Complexity

Phase	Time	Space
SEARCH_RUN (DFS)	O(V + E) where V = cells, E = open passages	O(V)
BFS path computation	O(V + E)	O(V)
SPEED_RUN (path follow)	O(path length)	O(1) extra

End of Lab 5 Report